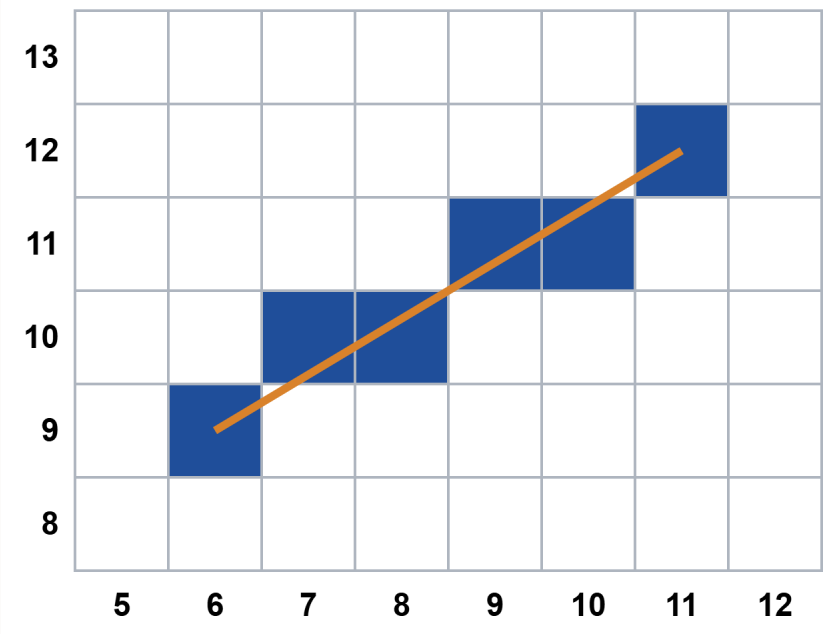
0613-403 · Week 2 · Lecture 3

The DDA Line Drawing Algorithm

Ekramul Haque Amin

Lecturer, AI and Data Science

Dhaka International University



Warm-up: three quick questions

Q1

What is the RGB value of yellow?

Q2

Scan convert the point (5.6, 2.2) by rounding.

Q3

Why does a steep line get gaps with the direct method?

Take one minute. Write your answers, then compare with your neighbour.

Recap: the direct method

How it works

For each x, compute y = mx + b and round it.

Line (2, 3) to (7, 6): pixels (2,3) (3,4) (4,4) (5,5) (6,5) (7,6)

Its two problems

- Slow: one multiplication for every pixel
- Steep lines (|m| > 1) get gaps

Today: the DDA algorithm removes the multiplication and handles every slope.

What is DDA?

Digital Differential Analyzer: an incremental line algorithm.

Incremental

Each new point is found from the previous one, not from scratch.

Small equal steps

Move one pixel at a time along the longer direction.

Additions only

Inside the loop there is no multiplication, only adding.

The key idea: reuse the previous y

At step i: $y_i = m \cdot x_i + b$

Next step: $y_{i+1} = m \cdot x_{i+1} + b$

$= m(x_i + \Delta x) + b$

$= (m \cdot x_i + b) + m \cdot \Delta x$

$= y_i + m \cdot \Delta x$

If $\Delta x = 1$: $y_{i+1} = y_i + m$

Each new y is the old y plus the slope. One addition replaces a multiplication.

Example: m = 0.6, start y = 9

9 → 9.6 → 10.2 → 10.8 → ...

Two cases, depending on the slope

Gentle slope: $|m| \le 1$

x changes faster, so step along x.

$$x_{i+1} = x_i + 1$$

$$y_{i+1} = y_i + m$$

Every column gets exactly one pixel.

Steep slope: $|m| > 1$

y changes faster, so step along y.

$$y_{i+1} = y_i + 1$$

$$x_{i+1} = x_i + 1/m$$

Every row gets exactly one pixel: no gaps.

Why 1/m? From $m = \Delta y/\Delta x$, with $\Delta y = 1$ we get $\Delta x = 1/m$.

The DDA algorithm (all slopes in one)

1. Input end points (x1, y1) and (x2, y2).
2. dx = x2 − x1, dy = y2 − y1
3. steps = max(|dx|, |dy|)
4. xInc = dx ÷ steps, yInc = dy ÷ steps
5. Start at x = x1, y = y1.
6. Repeat steps + 1 times: plot (round(x), round(y)); then x = x + xInc, y = y + yInc.

Why this works

steps = the longer side, so one of xInc or yInc is always ±1 and the other is the slope (or 1/slope).

It handles gentle, steep and negative slopes without separate cases.

How to solve a DDA question in the exam

1. Write dx, dy and steps = max(|dx|, |dy|).
2. Write xInc = dx ÷ steps and yInc = dy ÷ steps.
3. Draw a table: Step, x, y, Pixel.
4. Start from (x1, y1); add xInc and yInc on each row.
5. Round each x and y with floor(v + 0.5).
6. List the pixels, and plot them on a grid if asked.

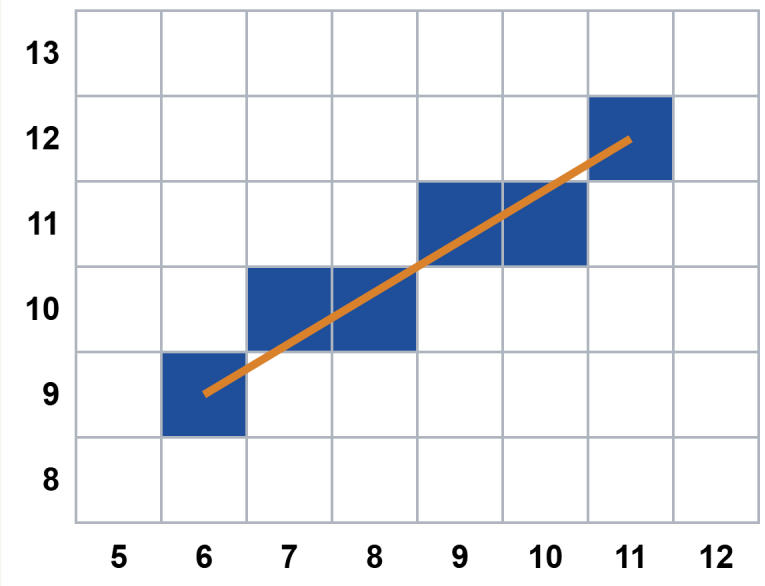
Check your answer

- There are steps + 1 pixels
- The first pixel is (x1, y1)
- The last pixel is (x2, y2)
- One coordinate changes by exactly 1 each row

Example 1: line from (6, 9) to (11, 12)

dx = 5, dy = 3, steps = 5, xInc = 1, yInc = 0.6

Step	x	y	Pixel (round x, round y)
0	6	9	(6, 9)
1	7	9.6	(7, 10)
2	8	10.2	(8, 10)
3	9	10.8	(9, 11)
4	10	11.4	(10, 11)
5	11	12	(11, 12)

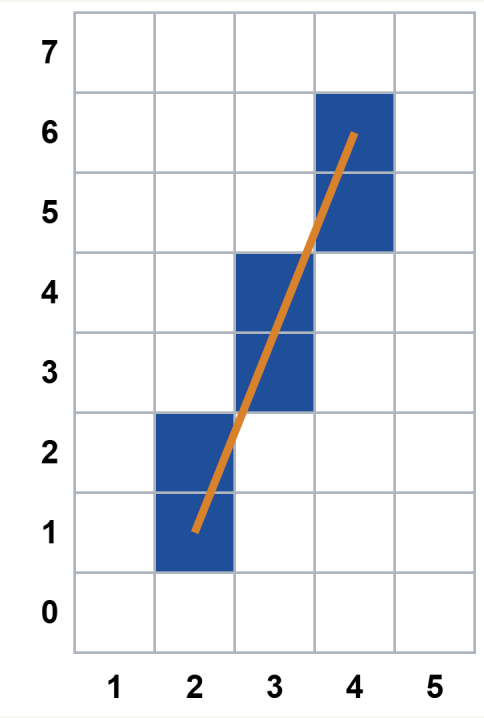


Example 2: a steep line, (2, 1) to (4, 6)

dx = 2, dy = 5, steps = 5, xInc = 0.4, yInc = 1

m = 2.5 > 1, so y goes up by 1 each step.

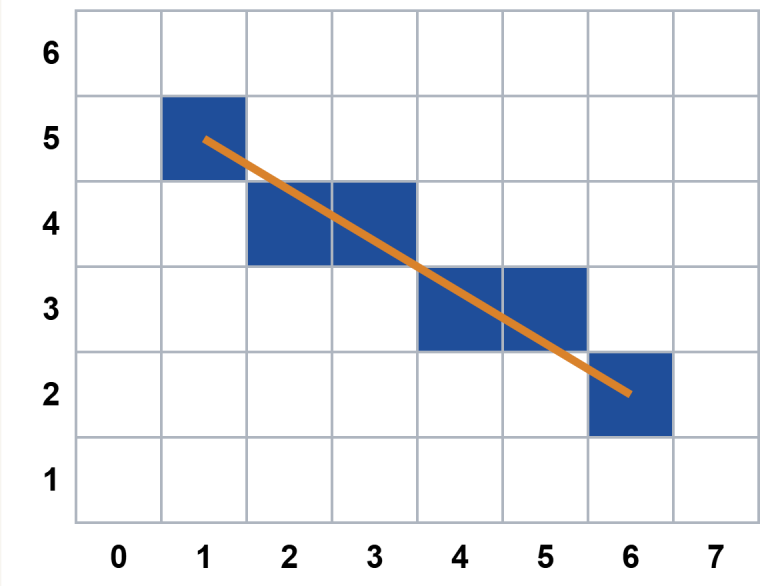
Step	x	y	Pixel (round x, round y)
0	2	1	(2, 1)
1	2.4	2	(2, 2)
2	2.8	3	(3, 3)
3	3.2	4	(3, 4)
4	3.6	5	(4, 5)
5	4	6	(4, 6)



Example 3: negative slope, (1, 5) to (6, 2)

dx = 5, dy = −3, steps = 5, xInc = 1, yInc = −0.6

Step	x	y	Pixel (round x, round y)
0	1	5	(1, 5)
1	2	4.4	(2, 4)
2	3	3.8	(3, 4)
3	4	3.2	(4, 3)
4	5	2.6	(5, 3)
5	6	2	(6, 2)



Special cases: DDA still works

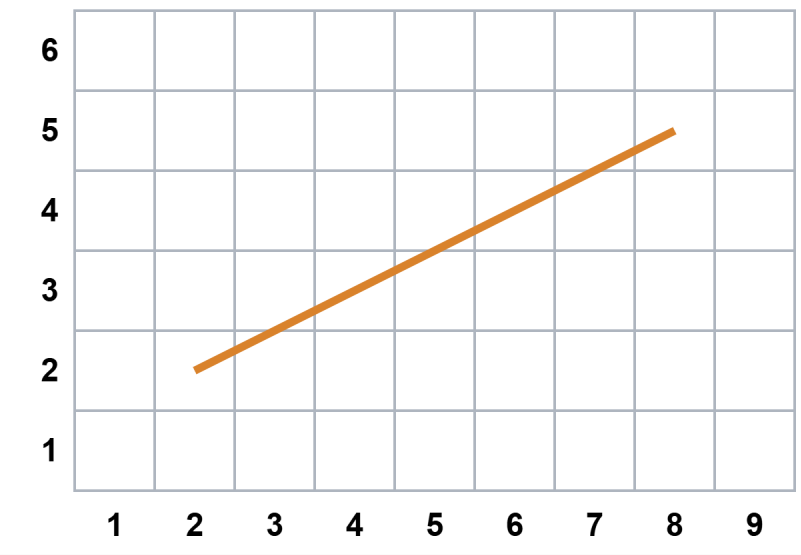
Case	Example	xInc, yInc	What happens
Horizontal line	(2, 3) to (7, 3)	1, 0	y never changes
Vertical line	(4, 1) to (4, 6)	0, 1	x never changes
45° diagonal	(1, 1) to (5, 5)	1, 1	No rounding needed
Right to left	(8, 5) to (2, 2)	−1, −0.5	Same pixels as (2, 2) to (8, 5)
Single point	(3, 3) to (3, 3)	steps = 0	Plot one pixel; do not divide by 0

Class exercise: your turn

Use DDA to scan convert the line from (2, 2) to (8, 5).

1. Find dx, dy, steps, xInc and yInc
2. Fill in the table
3. Shade the pixels on the grid

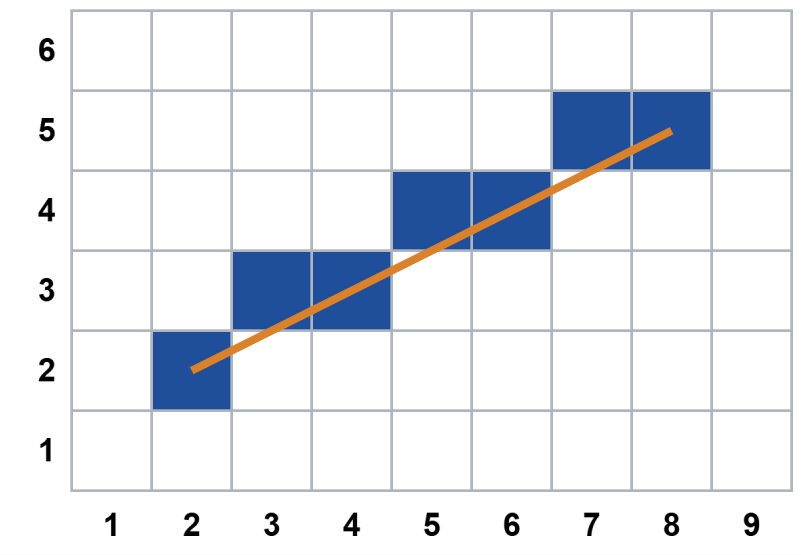
> **Time: 5 minutes. Work in pairs.**



Exercise answer: (2, 2) to (8, 5)

dx = 6, dy = 3, steps = 6, xInc = 1, yInc = 0.5

Step	x	y	Pixel (round x, round y)
0	2	2	(2, 2)
1	3	2.5	(3, 3)
2	4	3	(4, 3)
3	5	3.5	(5, 4)
4	6	4	(6, 4)
5	7	4.5	(7, 5)
6	8	5	(8, 5)



DDA in C++ with OpenGL

```
void ddaLine(int x1, int y1, int x2, int y2) {
    int dx = x2 - x1, dy = y2 - y1;
    int steps = max(abs(dx), abs(dy));
    float xInc = dx / (float) steps;
    float yInc = dy / (float) steps;
    float x = x1, y = y1;
    glBegin(GL_POINTS);
    for (int i = 0; i ⩽ steps; i++) {
        glVertex2i(round(x), round(y));
        x += xInc;  y += yInc;
    }
    glEnd();
}
```

Advantages and disadvantages

Advantages

- Faster than the direct method: no multiplication in the loop
- Simple to understand and code
- Works for every slope, with no gaps

Disadvantages

- Uses floating-point numbers, slower than integers
- Rounding at every step takes time
- Small errors add up on long lines, so pixels can drift

Next class: Bresenham's algorithm uses only integers and fixes these problems.

Direct method vs DDA

	Direct method	DDA
How y is found	y = mx + b every time	y = previous y + m
Work per pixel	Multiply, add, round	Add, round
Steep lines	Gaps unless handled separately	No gaps: steps along the longer side
Number type	Floating point	Floating point
Speed	Slow	Faster

Key points from today

1. DDA is incremental: $y_{i+1} = y_i + m$ when x steps by 1.
2. Step along the longer direction: steps = max(|dx|, |dy|).
3. xInc = dx ÷ steps and yInc = dy ÷ steps; round each point.
4. It works for gentle, steep and negative slopes.
5. It still uses floating point and rounding.

Practice questions

1. Use DDA to find the pixels of the line from (1, 1) to (6, 4).
2. Use DDA to find the pixels of the line from (3, 2) to (5, 8).
3. Use DDA to find the pixels of the line from (5, 1) to (2, 7).
4. Why is steps chosen as max(|dx|, |dy|)?
5. Why is DDA faster than the direct method? Give two of its drawbacks.

Coming up

Next lecture

Bresenham's line algorithm

Choosing pixels with a decision parameter, using integers only

This week's lab

Lab 2: DDA and Bresenham

Code both algorithms in Python with PyOpenGL and compare the lines they draw

Questions?

Thank you

Ekramul Haque Amin · Lecturer, AI and Data Science